

```

# CNN (MobileNet) for Plant Disease Detection:
#The goal of this study is to apply deep learning to diagnose plant leaf diseases.
#The goal is to use a pretrained MobileNet architecture to create an accurate and effective image categorization model.
#This model lowers manual inspection errors, increases agricultural productivity, and aids in the early detection of plant diseases.

from google.colab import drive
drive.mount('/content/drive')

🔗 Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

## Dataset Description

import zipfile
import os

# Enter the directory where the zip file should be extracted and its path
compressed_file = '/content/drive/MyDrive/Colab Notebooks/archive.zip'
destination_folder = '/content/'

# Opening the zip file and extracting all the required contents to the specified directory
with zipfile.ZipFile(compressed_file, 'r') as zip_handler:
    zip_handler.extractall(destination_folder)

# Importing the TensorFlow libraries
import tensorflow as tf
from tensorflow import keras

# Visualization libraries
import matplotlib.pyplot as plt
import numpy as np

# Operating the system utilities
import os

# dimensions of the image
img_dimension = 224
img_target_dim = (img_dimension, img_dimension)
img_input_shape = (img_dimension, img_dimension, 3)

# Training the parameters
train_batch_size = 32
num_epochs = 25

base_dir = "/content/New Plant Diseases Dataset(Augmented)/New Plant Diseases Dataset(Augmented)"
train_dir = os.path.join(base_dir, "train")
test_dir = os.path.join(base_dir, "valid")

## Augmenting Data
train_image_gen = keras.preprocessing.image.ImageDataGenerator(
    rescale=1/255.0,
    shear_range=0.2,
    zoom_range=0.2,
    width_shift_range=0.2,
    height_shift_range=0.2,
    fill_mode="nearest"
)

# data preprocessing the image for testing (no augmentation)
test_image_gen = keras.preprocessing.image.ImageDataGenerator(
    rescale=1/255.0
)

# Loading and augmenting training images from the directory
train_dataset = train_image_gen.flow_from_directory(
    train_dir,
    target_size=img_target_dim,
    batch_size=train_batch_size,
    class_mode="categorical"

```

```
)

# Without augmentation, Loading the test images from directory
test_dataset = test_image_gen.flow_from_directory(
    test_dir,
    target_size=img_target_dim,
    batch_size=train_batch_size,
    class_mode="categorical"
)
```

Found 70295 images belonging to 38 classes.
Found 17572 images belonging to 38 classes.

```
# class labels are extracted from the training dataset
class_labels = list(train_dataset.class_indices.keys())
print("Class Indices:", train_dataset.class_indices)
```

Indices: {'Apple__Apple_scab': 0, 'Apple__Black_rot': 1, 'Apple__Cedar_apple_rust': 2, 'Apple__healthy': 3, 'Blueberry__healthy': 4,

```
import json
```

```
# The class indices are saved in a JSON file.
with open('class_indices.json', 'w') as json_file:
    json.dump(train_dataset.class_indices, json_file)
```

```
# Providing a download link for the JSON file (Colab/Jupyter)
from IPython.display import FileLink
display(FileLink('class_indices.json'))
```

```
# Initializing the MobileNet pre-trained model (excluding top layers)
pretrained_base_model = tf.keras.applications.MobileNet(
    weights="imagenet",
    include_top=False,
    input_shape=img_input_shape
)
```

```
# Freezing the pre-trained model layers
pretrained_base_model.trainable = False
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet/mobilenet_1_0_224_tf_no_top.h5
17225924/17225924 — 0s 0us/step

```
# Defining the input layer
input_layer = keras.Input(shape=img_input_shape)
```

```
# Passing the inputs through the pretrained MobileNet base model
x = pretrained_base_model(input_layer, training=False)
```

```
# Adding Global Average Pooling layer
x = tf.keras.layers.GlobalAveragePooling2D()(x)
```

```
# Adding the Dropout for regularization
x = tf.keras.layers.Dropout(0.2)(x)
```

```
# Adding the final Dense layer for classification
output_layer = tf.keras.layers.Dense(
    len(class_labels),
    activation="softmax"
)(x)
```

```
# Defining the full model
plant_disease_model = keras.Model(
    inputs=input_layer,
    outputs=output_layer,
    name="LeafDisease_MobileNet"
)
```

```
# Defining the optimizer
model_optimizer = tf.keras.optimizers.Adam()
```

```
# Compiling the model
plant_disease_model.compile(
    optimizer=model_optimizer,
    loss=tf.keras.losses.CategoricalCrossentropy(from_logits=True),
    metrics=[
        keras.metrics.CategoricalAccuracy(),
        'accuracy'
    ]
)
```

```
# Training the model
training_history = plant_disease_model.fit(
    train_dataset,
    validation_data=test_dataset,
    epochs=num_epochs,
    steps_per_epoch=150,
    validation_steps=100
)
```

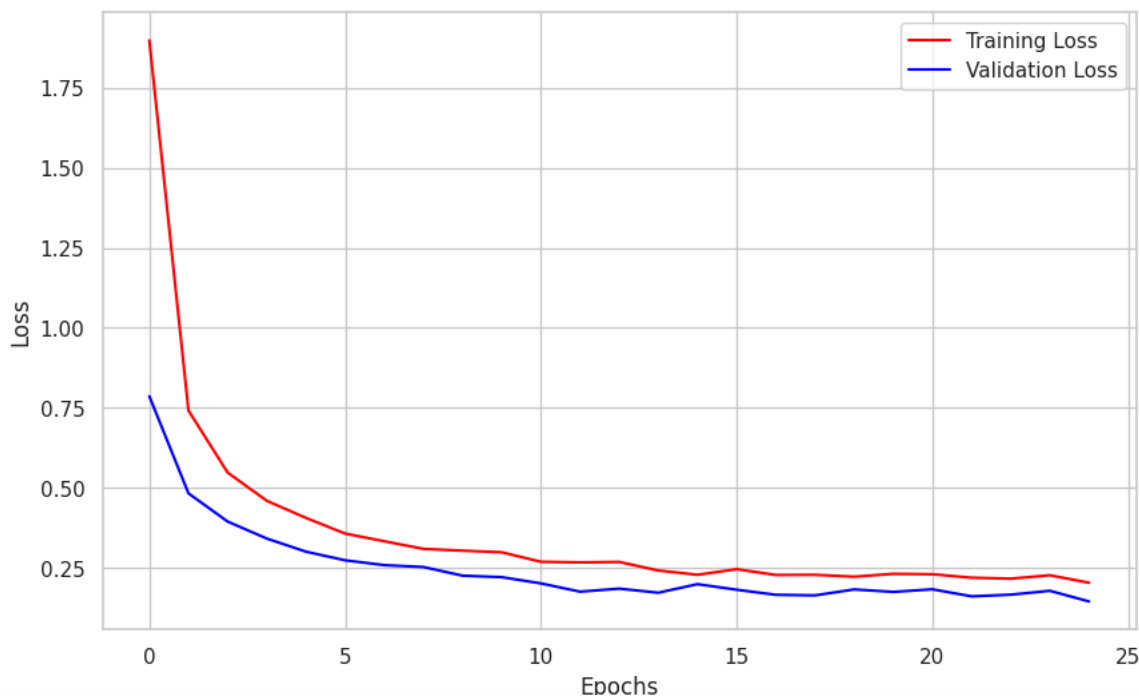
Epoch 1/25
150/150 ————— 75s 483ms/step - accuracy: 0.2960 - categorical_accuracy: 0.2960 - loss: 2.7695 - val_accuracy: 0.8147 - va
Epoch 2/25
150/150 ————— 71s 473ms/step - accuracy: 0.7720 - categorical_accuracy: 0.7720 - loss: 0.8222 - val_accuracy: 0.8625 - va
Epoch 3/25
150/150 ————— 70s 472ms/step - accuracy: 0.8295 - categorical_accuracy: 0.8295 - loss: 0.5855 - val_accuracy: 0.8856 - va
Epoch 4/25
150/150 ————— 69s 460ms/step - accuracy: 0.8597 - categorical_accuracy: 0.8597 - loss: 0.4883 - val_accuracy: 0.8991 - va
Epoch 5/25
150/150 ————— 68s 452ms/step - accuracy: 0.8750 - categorical_accuracy: 0.8750 - loss: 0.4239 - val_accuracy: 0.9062 - va
Epoch 6/25
150/150 ————— 63s 424ms/step - accuracy: 0.8915 - categorical_accuracy: 0.8915 - loss: 0.3782 - val_accuracy: 0.9131 - va
Epoch 7/25
150/150 ————— 63s 423ms/step - accuracy: 0.8942 - categorical_accuracy: 0.8942 - loss: 0.3527 - val_accuracy: 0.9159 - va
Epoch 8/25
150/150 ————— 63s 424ms/step - accuracy: 0.9145 - categorical_accuracy: 0.9145 - loss: 0.2907 - val_accuracy: 0.9206 - va
Epoch 9/25
150/150 ————— 63s 419ms/step - accuracy: 0.9004 - categorical_accuracy: 0.9004 - loss: 0.3110 - val_accuracy: 0.9322 - va
Epoch 10/25
150/150 ————— 63s 423ms/step - accuracy: 0.9139 - categorical_accuracy: 0.9139 - loss: 0.2884 - val_accuracy: 0.9294 - va
Epoch 11/25
150/150 ————— 63s 421ms/step - accuracy: 0.9196 - categorical_accuracy: 0.9196 - loss: 0.2790 - val_accuracy: 0.9366 - va
Epoch 12/25
150/150 ————— 64s 426ms/step - accuracy: 0.9124 - categorical_accuracy: 0.9124 - loss: 0.2739 - val_accuracy: 0.9447 - va
Epoch 13/25
150/150 ————— 64s 425ms/step - accuracy: 0.9107 - categorical_accuracy: 0.9107 - loss: 0.2782 - val_accuracy: 0.9428 - va
Epoch 14/25
150/150 ————— 64s 425ms/step - accuracy: 0.9180 - categorical_accuracy: 0.9180 - loss: 0.2542 - val_accuracy: 0.9444 - va
Epoch 15/25
150/150 ————— 43s 291ms/step - accuracy: 0.9295 - categorical_accuracy: 0.9295 - loss: 0.2265 - val_accuracy: 0.9334 - va
Epoch 16/25
150/150 ————— 71s 476ms/step - accuracy: 0.9277 - categorical_accuracy: 0.9277 - loss: 0.2426 - val_accuracy: 0.9341 - va
Epoch 17/25
150/150 ————— 71s 474ms/step - accuracy: 0.9249 - categorical_accuracy: 0.9249 - loss: 0.2291 - val_accuracy: 0.9466 - va
Epoch 18/25
150/150 ————— 70s 470ms/step - accuracy: 0.9274 - categorical_accuracy: 0.9274 - loss: 0.2299 - val_accuracy: 0.9466 - va
Epoch 19/25
150/150 ————— 69s 463ms/step - accuracy: 0.9283 - categorical_accuracy: 0.9283 - loss: 0.2202 - val_accuracy: 0.9384 - va
Epoch 20/25
150/150 ————— 67s 448ms/step - accuracy: 0.9219 - categorical_accuracy: 0.9219 - loss: 0.2370 - val_accuracy: 0.9438 - va
Epoch 21/25
150/150 ————— 63s 420ms/step - accuracy: 0.9213 - categorical_accuracy: 0.9213 - loss: 0.2385 - val_accuracy: 0.9356 - va
Epoch 22/25
150/150 ————— 63s 424ms/step - accuracy: 0.9213 - categorical_accuracy: 0.9213 - loss: 0.2381 - val_accuracy: 0.9466 - va
Epoch 23/25
150/150 ————— 63s 424ms/step - accuracy: 0.9226 - categorical_accuracy: 0.9226 - loss: 0.2178 - val_accuracy: 0.9413 - va
Epoch 24/25
150/150 ————— 64s 426ms/step - accuracy: 0.9164 - categorical_accuracy: 0.9164 - loss: 0.2404 - val_accuracy: 0.9391 - va
Epoch 25/25
150/150 ————— 64s 427ms/step - accuracy: 0.9409 - categorical_accuracy: 0.9409 - loss: 0.1951 - val_accuracy: 0.9516 - va

```
# Extracting the loss values
training_loss = training_history.history['loss']
validation_loss = training_history.history['val_loss']
```

```
# Defining the range of epochs
epoch_range = range(len(training_loss))

# Plotting the training and validation loss
fig = plt.figure(figsize=(10, 6))
plt.plot(epoch_range, training_loss, c="red", label="Training Loss")
plt.plot(epoch_range, validation_loss, c="blue", label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
```

 <matplotlib.legend.Legend at 0x79ace4468f50>

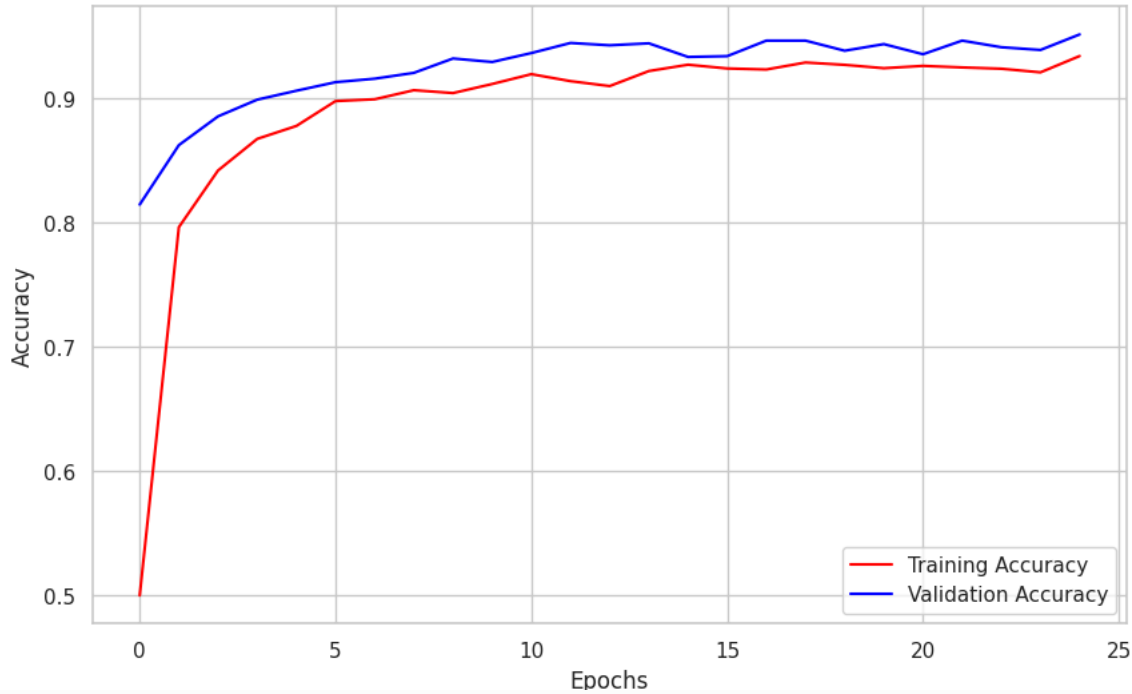


```
# Extracting the accuracy values
training_accuracy = training_history.history['categorical_accuracy']
validation_accuracy = training_history.history['val_categorical_accuracy']

# Defining the range of epochs
epoch_range_acc = range(len(training_accuracy))

# Plotting the training and validation accuracy
fig = plt.figure(figsize=(10, 6))
plt.plot(epoch_range_acc, training_accuracy, c="red", label="Training Accuracy")
plt.plot(epoch_range_acc, validation_accuracy, c="blue", label="Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
```

 <matplotlib.legend.Legend at 0x79acefb210>



```
# Using the.keras extension to save the trained model
plant_disease_model.save('plant_disease.keras')
```

```
import tensorflow as tf
```

```
# The trained model should be converted to TensorFlow Lite format
model_converter = tf.lite.TFLiteConverter.from_keras_model(plant_disease_model)
tflite_converted_model = model_converter.convert()
```

```
# TensorFlow Lite model saving it in to a file
with open('plant_disease_model.tflite', 'wb') as file:
    file.write(tflite_converted_model)
```

 Saved artifact at '/tmp/tmpkyu4ild3'. The following endpoints are available:

```
* Endpoint 'serve'
args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name='keras_tensor_86')
Output Type:
```

```
TensorSpec(shape=(None, 38), dtype=tf.float32, name=None)
```

```
Captures:
```

```
133789908326288: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908327440: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908327824: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908327056: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908326096: TensorSpec(shape=(), dtype=tf.resource, name=None)
133787148806160: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908326672: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908325136: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908328400: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908327632: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908328784: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908330320: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908326480: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908327248: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908329360: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908333008: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908334352: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908334736: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908334544: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908333200: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908335888: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908336272: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445421008: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908335504: TensorSpec(shape=(), dtype=tf.resource, name=None)
133789908333584: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445422160: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445422544: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
133790445422928: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445422736: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445421392: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445424080: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445424464: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445424848: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445424656: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445421584: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445426000: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445426384: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445426768: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445426576: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445420816: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445423696: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445428496: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445428880: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445428688: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445425232: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445427536: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445430608: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445430992: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445430800: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445427920: TensorSpec(shape=(), dtype=tf.resource, name=None)
133790445432144: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.